

1 TAJIR CHAIN TESTNET

1.1 Black-Box Penetration Testing Report

Strategic Security Summary – Follow-up Report
June 09, 2026

Prepared by: Eddie – Cybersecurity Consultant
Contact: Security@TajirChain.com

1.1.1 1. Relation to Previous Reports

This report serves as a technical follow-up to the **Black-Box Penetration Testing Report – Bridge Security Assessment – Follow-up Report (June 06, 2026)**.

The June 06 engagement confirmed measurable operational improvement in RPC stability and correct EIP-1967 Proxy + Lock & Mint transaction behavior. This report presents additional high-severity findings identified through continued black-box testing, automated scanning, and manual verification between June 07–09, 2026.

All findings are derived exclusively from public interfaces, browser DevTools, JavaScript bundle analysis, network traffic inspection, and automated vulnerability scanning.

1.1.2 2. Executive Summary

Black-box penetration testing conducted between June 06–09, 2026, confirmed measurable operational improvements in the Tajir Chain Testnet Bridge, particularly in RPC stability and correct Lock & Mint transaction behavior. However, the assessment also revealed several critical and high-severity security issues that pose significant risk if the platform progresses toward mainnet deployment or handles real value.

The most critical findings stem from extensive client-side exposure of sensitive configuration data through VITE environment variables. This includes direct leakage of smart contract addresses (Bridge, Proof of Efficiency, and Rollup Manager), internal RPC proxy endpoints, and multiple disabled security-related feature flags. Compounding this risk is a clear WalletConnect misconfiguration, where a placeholder Project ID (“YOUR_PROJECT_ID”) remains in the production JavaScript bundle alongside an excessively large list of supported chains.

Additionally, the faucet exhibits serious weaknesses, including the absence of server-side CAPTCHA enforcement, overly aggressive rate limiting that results in a ~24-hour cooldown after minimal usage, and leakage of internal infrastructure details. Several undocumented /v1/ API endpoints were also discovered, expanding the attack surface without proper authentication or rate limiting controls.

Overall Risk Posture: HIGH

While the bridge demonstrates functional correctness in its core cross-chain mechanism, the current security posture is undermined by preventable client-side exposures and insufficient protection mechanisms on public-facing components. Immediate remediation is strongly recommended before any progression to mainnet or increased usage.

1.1.3 3. Key Technical Findings

1.1.3.1 3.1 High Risk Findings 3.1.1 Frontend Exposure via VITE Environment Variable Leaks

Severity: Critical

Description:

The production frontend build exposes multiple sensitive VITE environment variables. These variables contain critical infrastructure details including main smart contract addresses (Bridge Contract, Proof of Efficiency - POE, and Rollup Manager), internal RPC proxy endpoints (/l1rpc, /l2rpc, /bridgeservice), and configuration flags that control security and user experience features.

Notably, the flag `VITE_ETHEREUM_FORCE_UPDATE_GLOBAL_EXIT_ROOT` is set to `true`. Additionally, several security-related feature flags are disabled: - `VITE_ENABLE_DEPOSIT_WARNING: false` - `VITE_ENABLE_REPORT_FORM: false` - `VITE_ENABLE_OUTDATED_NETWORK_MODAL: false`

Discovery Method:

Analysis of browser DevTools Sources tab and inspection of built JavaScript bundles, combined with automated scanning for exposed environment variables.

Potential Security Impact:

Attackers can directly obtain smart contract addresses, discover internal service endpoints, and identify disabled security controls. This significantly reduces the effort required to perform targeted attacks against the bridge and related components.

Contract Addresses Extracted from Production JavaScript Bundle

During detailed analysis of the minified production JavaScript bundle (`index-B8GZX019.js`), the following critical smart contract addresses were directly extracted from client-side code: - `VITE_ETHEREUM_BRIDGE_CONTRACT_ADDRESS: 0x528e26b25a34a4a5d0bdba1d57d318153d2ed582` - `VITE_POLYGON_ZK_EVM_BRIDGE_CONTRACT_ADDRESS: 0x528e26b25a34a4A5d0dbDa1d57D318153d2ED582`

This confirms that the exact on-chain addresses for the Lock & Mint bridge mechanism are fully exposed in the production frontend bundle, significantly reducing the effort required for targeted attacks against the bridge contracts.

Evidence from Browser DevTools (Sources Tab):

The following screenshot from `bridge.testnet.tajirchain.com` DevTools Sources tab shows the exposed VITE environment variables in the built JavaScript bundle:

Key Exposed Sensitive Variables (from the screenshot): - `VITE_ETHEREUM_BRIDGE_CONTRACT_ADDRESS` - `VITE_ETHEREUM_PROOF_OF_EFFICIENCY_CONTRACT_ADDRESS` - `VITE_ETHEREUM_ROLLUP_MANAGER_ADDRESS` - `VITE_POLYGON_ZK_EVM_BRIDGE_CONTRACT_ADDRESS` - `VITE_BRIDGE_API_URL` (pointing to internal `/bridgeservice`) - `VITE_POLYGON_ZK_EVM_RPC_URL` (pointing to internal `/l2rpc`) - `VITE_ETHEREUM_FORCE_UPDATE_GLOBAL_EXIT_ROOT: true` - Multiple disabled security/user-experience flags (`VITE_ENABLE_DEPOSIT_WARNING`, `VITE_ENABLE_REPORT_FORM`, `VITE_ENABLE_OUTDATED_NETWORK_MODAL`, etc.)

3.1.2 WalletConnect Misconfiguration

Severity: High

Description:

During analysis of the bundled JavaScript file (`index-B8GZX019.js`) of the Tajir Chain Bridge interface, a clear and direct misconfiguration was discovered in the WalletConnect integration:

- The following code was found inside the bundled file:

```
projectId: "YOUR_PROJECT_ID",
chains: [T],
rpcMap: { [T]: C.provider.connection.url },
showQrModal: 10
```

- The default placeholder value `"YOUR_PROJECT_ID"` was used instead of a real Project ID from WalletConnect Cloud.
- The application uses WalletConnect SDK v2 extensively, and the `projectId` is passed in multiple critical operations (client initialization, request validation, event emission, and Relay communication).

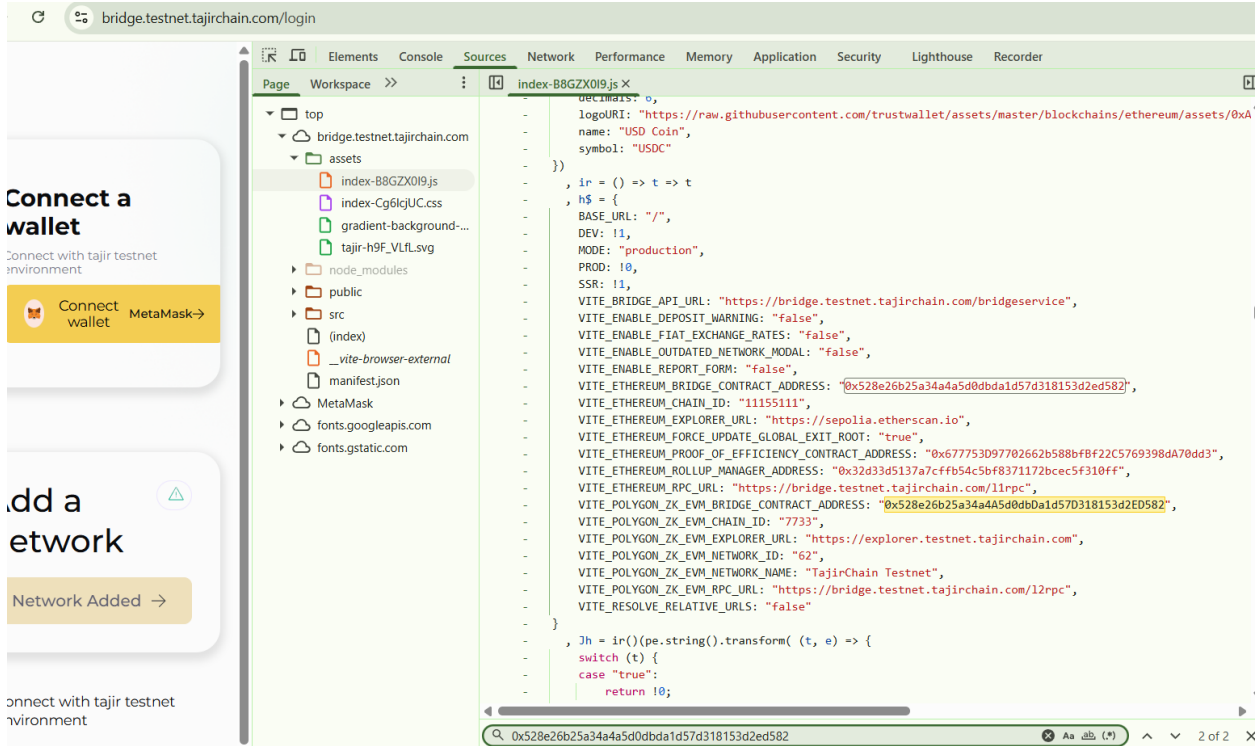


Figure 1: VITE Environment Variables Exposure

Discovery Method:

Inspection of the file `index-LjUIHZ2g.js` using browser Developer Tools (DevTools → Sources) and searching for `projectId` and `YOUR_PROJECT_ID`.

Evidence:

The following screenshot from DevTools Sources tab confirms the presence of the placeholder `projectId`: "`YOUR_PROJECT_ID`" inside the bundled JavaScript file:

Potential Security Impact:

This error renders WalletConnect features (such as the QR Modal and verification) ineffective or behaving unexpectedly. It also gives attackers clear information about how the application connects to wallets and the internal structure of the chains configuration.

3.1.3 Faucet Security Weaknesses and Internal Infrastructure Leakage

Severity: High

Description:

The faucet lacks server-side CAPTCHA enforcement. Rate limiting is insufficient and can be bypassed. Additionally, internal infrastructure details are leaked, including references to services such as `proxyd.tajir-12.svc:8545`. There is also evidence suggesting potential enumeration of hot wallet addresses.

Discovery Method:

Manual testing of the faucet claim flow, analysis of associated API requests, and inspection of error messages and network responses.

Evidence:

The following screenshot shows the faucet interface (`faucet.testnet.tajirchain.com`) and the Network tab during a claim request. Note the direct `POST /api/claim` request with a successful 200 response and the

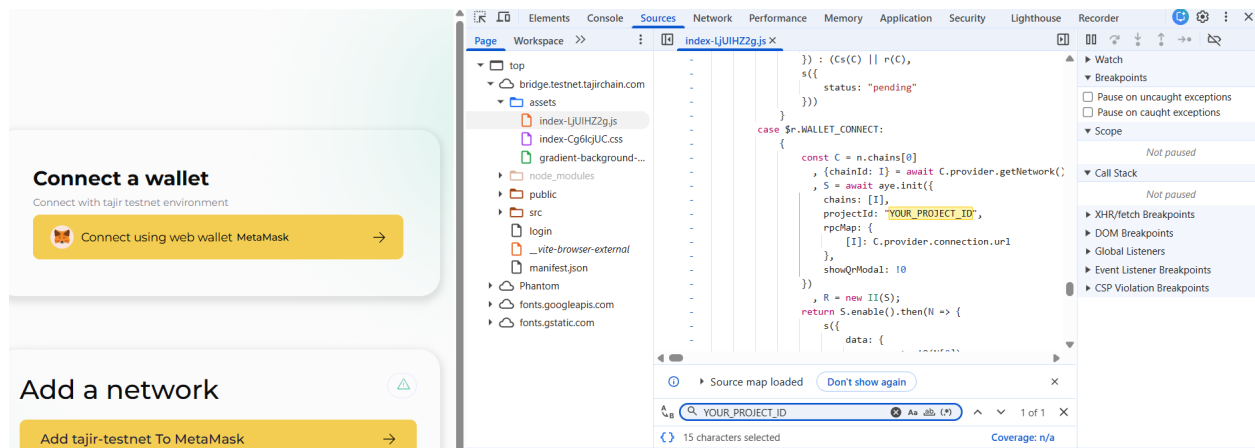


Figure 2: WalletConnect Project ID Placeholder Evidence

absence of any CAPTCHA challenge in the flow:

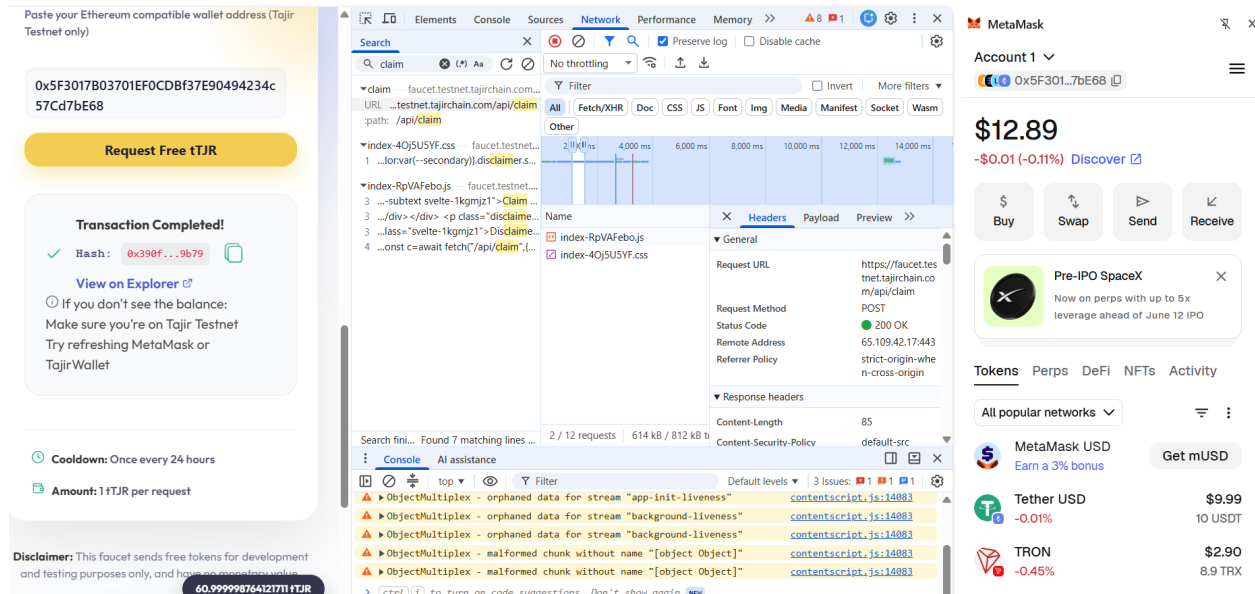


Figure 3: Faucet No CAPTCHA Evidence

Potential Security Impact:

Increased risk of faucet abuse for automated token draining and exposure of internal service architecture, which could aid further reconnaissance and targeted attacks.

3.1.4 Undocumented /v1/ API Surface Exposure

Severity: High

Description:

Multiple undocumented endpoints under the /v1/ path were discovered and tested. These include endpoints related to conversion (/v1/convert/*) and onramp functionality (/v1/onramp/*). Several of these endpoints lack proper authentication, rate limiting, or input validation.

Discovery Method:

Network traffic analysis during normal Bridge operations combined with targeted endpoint enumeration and fuzzing.

Potential Security Impact:

Potential for information disclosure, abuse of transaction building and pricing functions, and increased attack surface for denial-of-service or business logic attacks.

1.1.3.2 3.2 Medium Risk Findings 3.2.1 Weak Security Headers and Cookie Misconfigurations Severity: Medium-High

Weak or missing security headers were observed across Bridge, Faucet, and Explorer domains. Content-Security-Policy includes unsafe directives (`unsafe-inline`, `unsafe-eval`). The `security.txt` file returns SPA fallback content instead of proper security policy information. Cookie attributes lack proper security flags (e.g., missing `Secure` or `HttpOnly` in some cases).

3.2.2 Disabled Feature Flags and Internal References Severity: Medium

Multiple feature flags that control security warnings, reporting forms, and network status modals are disabled in production. Internal service references remain visible in frontend code, providing additional reconnaissance information to attackers.

1.1.4 4. Positive Technical Observations

The following improvements were validated: - Significant improvement in **RPC stability** since the June 05 testing round. - Sampled transactions correctly follow the **EIP-1967 Proxy pattern** with Lock & Mint semantics and emit the expected **BridgeEvent**. - No unsafe delegatecall to EOA was observed in the analyzed transaction call stacks.

1.1.5 5. Visual Summary – Key Discoveries & Critical Bridge Risks

The following visual summary highlights the key discoveries from our black-box assessment alongside the most critical common risks associated with Lock & Mint bridge architectures.

Key Takeaway from the Visual Assessment:

Our testing confirmed multiple high-risk issues on the left side (Frontend Exposure, WalletConnect Misconfiguration, Faucet Weaknesses, Undocumented API Surface). The right side represents standard high-risk areas for Lock & Mint bridges that require dedicated future testing (Oracle manipulation, Proxy attacks, Double Spend vectors).

1.1.6 7. Recommendations for Usability and Security Enhancement

1.1.6.1 7.1 Usability Improvements

#	Recommended Feature	Description	Expected Benefit	Priority
1	Smart & Graduated Cooldown System	Replace the rigid 24-hour cooldown with a progressive system (e.g. 5 min → 30 min → 2 hours) combined with optional wallet signature verification.	Significantly improves user experience while maintaining abuse prevention.	High
2	Clear Faucet Status & Countdown	Display a real-time countdown timer and clear status messages instead of vague error messages.	Reduces user frustration and support requests.	High

#	Recommended Feature	Description	Expected Benefit	Priority
3	Proper WalletConnect Integration	Replace the placeholder <code>projectId: "YOUR_PROJECT_ID"</code> with a valid WalletConnect Cloud Project ID and restrict supported chains.	Enables full WalletConnect functionality (QR code + mobile wallets).	High
4	Enable Security & UX Feature Flags	Activate the currently disabled flags: <code>VITE_ENABLE_DEPOSIT_WARNING</code> , <code>VITE_ENABLE_REPORT_FORM</code> , and <code>VITE_ENABLE_OUTDATED_NETWORK_MODAL</code> .	Provides users with important warnings and feedback channels.	Medium
5	Improved Error Messages	Make error messages (especially rate limit errors) more user-friendly and actionable.	Enhances user trust and reduces confusion.	Medium

Console Errors and Event Listener Management Issues

During testing, repeated browser console warnings were observed, including: - `MaxListenersExceededWarning: Possible EventEmitter memory leak detected - ObjectMultiplex - orphaned data for stream and malformed chunk without name`

Although these issues are not directly exploitable security vulnerabilities, they indicate poor event listener management and potential memory leaks in the frontend application. This can negatively affect long-term stability and user experience. It is recommended to review and properly manage event listeners throughout the frontend codebase.

1.1.6.2 7.2 Security Hardening Recommendations

#	Recommended Feature	Description	Security Benefit	Priority
1	Server-Side CAPTCHA Enforcement	Implement CAPTCHA validation on the server side (preferably Cloudflare Turnstile or hCaptcha) for all faucet claim requests.	Effectively prevents automated abuse and token draining.	Critical
2	Advanced Multi-Factor Rate Limiting	Move from simple IP-based rate limiting to a system using Wallet Address + IP + Device Fingerprinting.	Significantly reduces bypass potential and faucet abuse.	High
3	Hide Sensitive Environment Variables	Remove or minimize exposure of sensitive <code>VITE_</code> variables (contract addresses, internal RPCs, feature flags) from the client-side JavaScript bundle.	Reduces reconnaissance surface and prevents direct leakage of critical infrastructure.	Critical
4	Strengthen Security Headers	Enforce strict Content-Security-Policy (without <code>unsafe-inline/unsafe-eval</code>), X-Frame-Options, and HSTS across all domains.	Protects against XSS, clickjacking, and other client-side attacks.	High
5	Secure and Document <code>/v1/</code> API Endpoints	Add proper authentication, rate limiting, and input validation to all undocumented <code>/v1/</code> endpoints (<code>/v1/convert/</code> , <code>/v1/onramp/</code> , etc.).	Reduces attack surface and prevents abuse of business logic functions.	High

Recommended				
#	Feature	Description	Security Benefit	Priority
6	Fix WalletConnect Configuration	Replace the placeholder Project ID with a legitimate one and limit supported chains to only officially supported networks.	Prevents misconfiguration and improves wallet connection security.	High
7	Prevent Internal Infrastructure Leakage	Sanitize API responses to remove internal service names (e.g. proxyd.tajir-l2.svc).	Limits attacker reconnaissance capabilities.	Medium-High

1.1.6.3 Summary of Recommendations by Priority The following table summarizes the top 5 highest priority recommendations (Critical and High) that should be addressed first:

Rank	Recommendation	Category	Expected Impact
1	Server-Side CAPTCHA Enforcement	Security	Prevents automated faucet abuse (Critical)
2	Hide Sensitive VITE Environment Variables	Security	Reduces reconnaissance and infrastructure leakage (Critical)
3	Fix WalletConnect Configuration	Security/UX	Enables full wallet functionality and improves security (High)
4	Advanced Multi-Factor Rate Limiting	Security	Significantly reduces faucet abuse and bypass (High)
5	Smart & Graduated Cooldown System	Usability	Greatly improves user experience while maintaining protection (High)

1.1.7 8. Conclusion

Black-box testing between June 06–09, 2026 confirmed operational improvements in the Bridge interface while revealing several critical and high-severity security issues. The most significant risks stem from extensive frontend secret exposure through VITE variables, WalletConnect misconfiguration, faucet weaknesses, and an undocumented API surface.

All findings are presented factually to assist the Tajir Chain team in prioritizing remediation. This report is a follow-up to the June 06, 2026 Black-Box Penetration Testing engagement and is intended solely for internal use by the technical team.

Security Analysis by Eddie | Security@TajirChain.com

This report is a follow-up to the Black-Box Penetration Testing Report dated June 06, 2026. All evidence and testing artifacts remain available for internal review by the Tajir Chain team.

End of Report

Tajir Chain Testnet Bridge - Final Security Assessment (June 2026)

Our Key Discoveries

- HIGH**
Frontend Exposure
(VITE_Leaks, RPC Proxies, Contract Addresses)
- HIGH**
WalletConnect Misconfiguration
(Project ID, Large Chains)
- HIGH**
Faucet Security Weaknesses
(No CAPTCHA, Rate Limiting, Internal RPC Leak)
- MEDIUM-HIGH**
Undocumented /v1/ API Surface
- MEDIUM**
Weak CSP & Missing Security Headers

Most Critical Bridge Risks

Most Critical Common Bridge Risks

- CRITICAL**
EIP-1967 Proxy Vulnerabilities
- CRITICAL**
Lock & Mint Mechanism Risks
(Double Spend, Replay, Inconsistent State)
- HIGH**
Validation & Oracle Manipulation
- HIGH**
API & RPC Surface Attacks
- CRITICAL**
Multisig / Validator Compromise

Figure 4: Tajir Chain Testnet Bridge - Final Security Assessment Mind Map